

Sharp Fade Barbershop

A Full Django Business Website with Online Appointment Scheduling

SEN 310 — Software Engineering Project Documentation
Federal University of Technology, Owerri (FUTO)

Document contents:	User Story, Use Case Diagram, Sequence Diagram, Class Diagram
Technology:	Python 3, Django Web Framework
Site pages:	Home, About, Services, Book Appointment, Contact, Schedule

1. User Story

1.1 Project Overview

Sharp Fade Barbershop is a full business website built with Django for a local barbershop. Beyond a simple booking form, the site gives the shop an online presence: a Home page that introduces the business, an About page describing the shop and its barbers, a Services page listing haircuts and prices, a Contact page for enquiries, and a Book Appointment flow that lets clients reserve a time slot with a chosen barber. The shop owner also has a Schedule page to see and manage every booking.

1.2 Primary Users

Client — a member of the public browsing the site, checking prices, and booking a haircut.

Shop Owner / Barber — manages the day's bookings and the shop's data (services, barbers).

1.3 User Stories

ID	User Story
US-01	As a client, I want to see a welcoming home page with a summary of top services, so that I get a quick feel for the shop.
US-02	As a client, I want to read an About page with barber profiles, so that I can choose someone whose style I trust.
US-03	As a client, I want to view all services with prices and durations, so that I know what I am paying for before I book.
US-04	As a client, I want to book an appointment by picking a service, barber, date and time, so that I can reserve my slot with a barber.
US-05	As a client, I want to send a message through a Contact page, so that I can ask a question without needing a phone call.
US-06	As a shop owner, I want to see every booked appointment in one schedule view, so that I know who is coming in and when.
US-07	As a shop owner, I want to cancel an appointment with a confirmation step, so that I don't accidentally remove a valid booking.
US-08	As a shop owner, I want to manage services, barbers, and bookings from Django's admin panel, so that I have a reliable way to manage the shop's data.

1.4 Acceptance Criteria (summary)

- Home, About, Services, and Contact pages must be reachable from the navigation bar on every page.
- A booking cannot be saved unless client name, phone, service, date, and time are all provided.
- A booking confirmation page must show the exact service, barber, date, and time saved.
- Cancelling an appointment must show a confirmation step before the record is removed.
- The Contact form must save the message and show a success notice without leaving the page flow.

2. Use Case Diagram

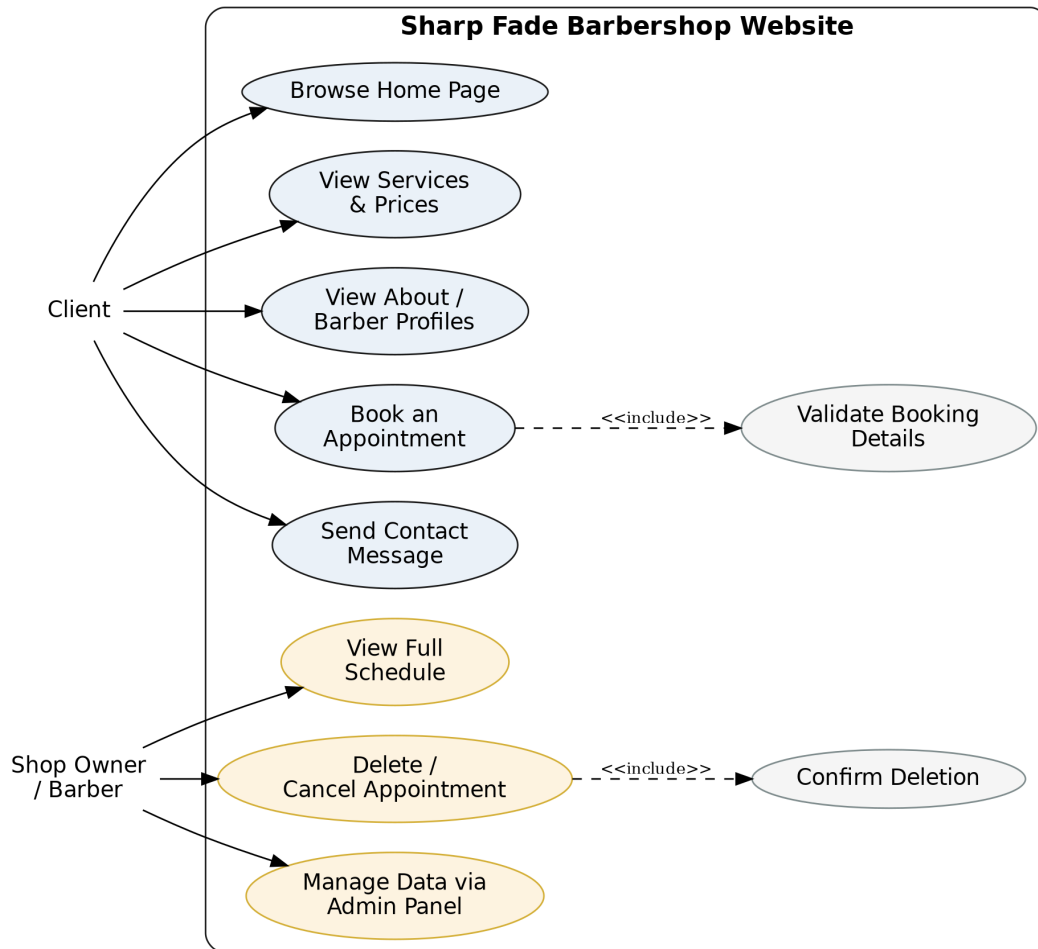


Figure 1: Use Case Diagram — Sharp Fade Barbershop Website

2.1 Actors

Client — the general public visitor who browses the site and books appointments.

Shop Owner / Barber — the business-side user who views the schedule, cancels bookings, and manages shop data.

2.2 Use Case Descriptions

Browse Home Page / View Services / View About

The client explores the site's informational pages: a home page with a summary of featured services, a full services list with prices and durations, and an about page with the shop's story and barber profiles. No login is required for any of these.

Book an Appointment

The client fills in a form with their name, phone, email, chosen service, preferred barber, date, and time. The system validates the details (*Validate Booking Details*) before saving the booking and

redirecting to a confirmation page.

Send Contact Message

The client submits a name, email, and message through the Contact page. The system saves it as a `ContactMessage` record and shows a success notice.

View Full Schedule / Delete-Cancel Appointment

The shop owner opens the Schedule page to see every booking in the system. They can cancel any booking, which first shows a confirmation step (*Confirm Deletion*) before the record is permanently removed.

Manage Data via Admin Panel

As a secondary path, the shop owner can log into Django's built-in admin interface to add or edit services and barbers, or make bulk corrections to bookings.

3. Sequence Diagram

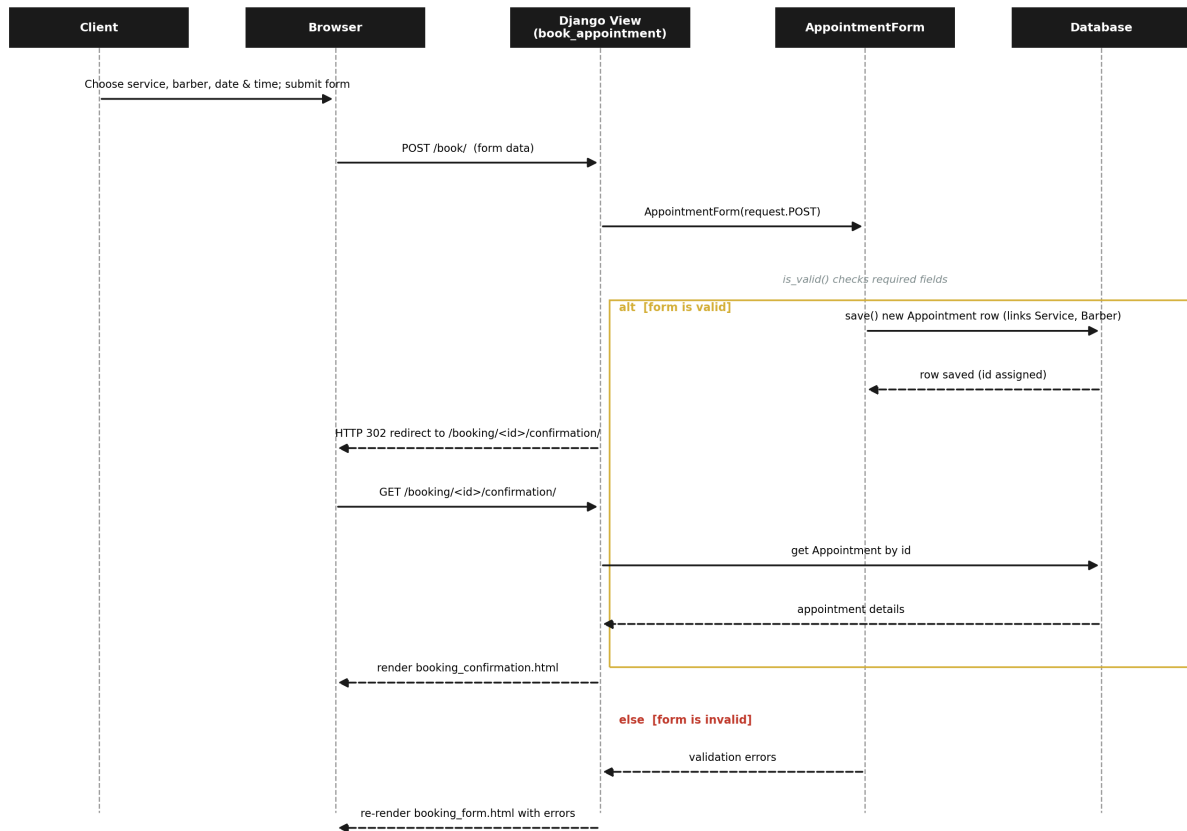


Figure 2: Sequence Diagram — Booking an Appointment

3.1 Scenario Described

This diagram traces the **Book an Appointment** use case end to end: from the client choosing a service, barber, date, and time in their browser, through Django's form-validation layer, to the database, and back to a confirmation page.

3.2 Step-by-Step Description

1. The Client selects a service, barber, date, and time on the booking form and submits it.
2. The Browser sends the form data as an HTTP POST request to the book_appointment view at /book/.
3. The view wraps the submitted data in an AppointmentForm instance and calls is_valid() to check it.
4. If the form is valid: the form's save() method writes a new Appointment row to the Database, linking it to the chosen Service and Barber. The Database confirms the row was saved with a new ID. The view redirects to the booking confirmation page, the Browser follows that redirect with a GET request, the view fetches the appointment by its ID, and the confirmation page is rendered

showing the booking details back to the client.

5. If the form is invalid: the form returns validation errors to the view, and the view re-renders the booking form with those errors instead of saving anything.

4. Class Diagram

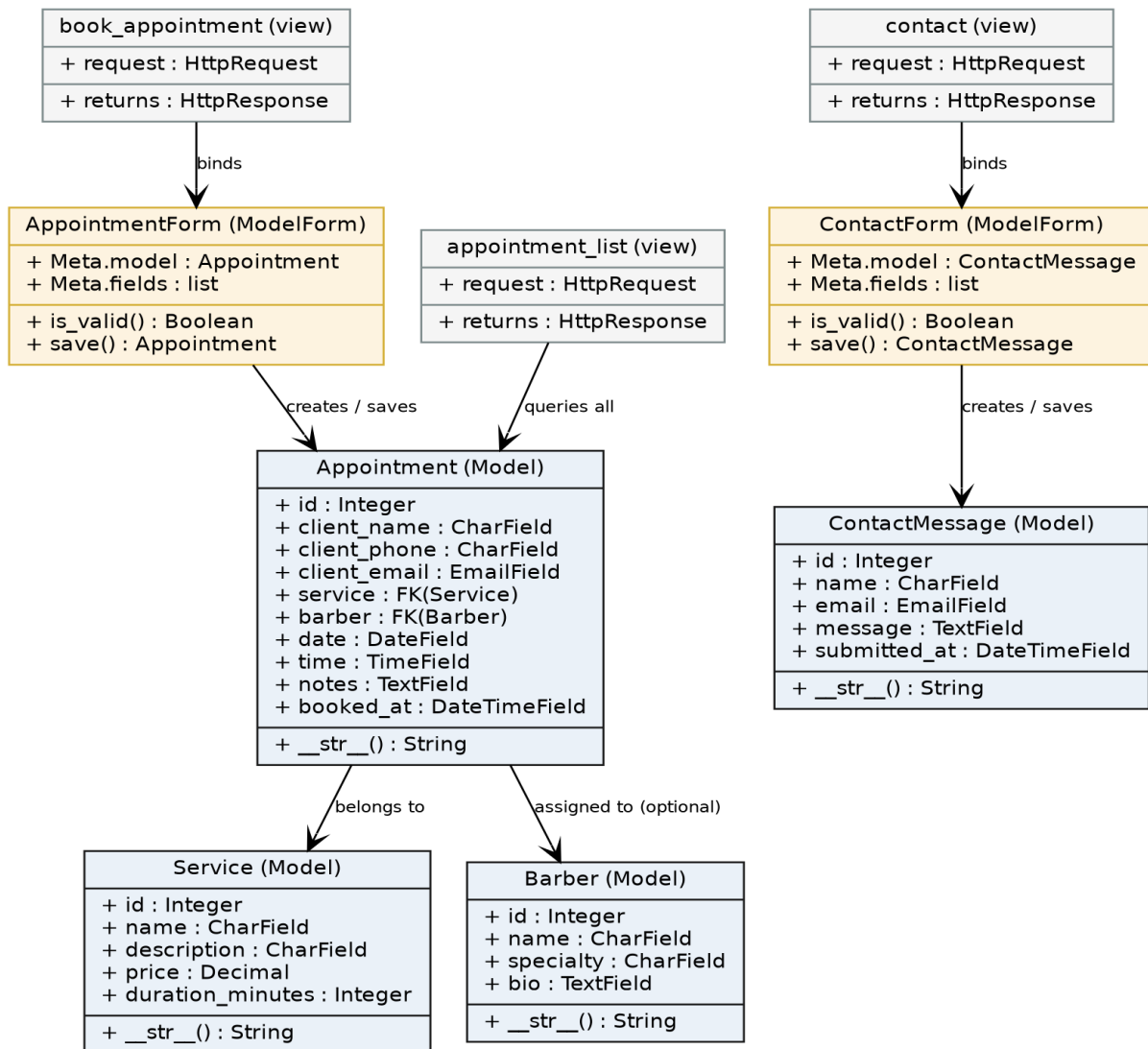


Figure 3: Class Diagram — Sharp Fade Barbershop Website

4.1 Classes and Responsibilities

Service (Model)

Represents a haircut or grooming service, storing its name, description, price, and duration. Displayed on the Home and Services pages.

Barber (Model)

Represents a barber working at the shop, with a name, specialty, and short bio. Displayed on the About page and offered as an optional choice when booking.

Appointment (Model)

The core booking record. Stores client contact details, links to the chosen Service and (optionally) Barber, plus the date, time, and any notes.

ContactMessage (Model)

Stores a message submitted through the Contact page: sender name, email, message body, and submission timestamp.

AppointmentForm / ContactForm (ModelForm)

Django ModelForms that validate and save Appointment and ContactMessage records respectively, keeping validation logic out of the views.

book_appointment / contact / appointment_list (views)

View functions that bind incoming form data, trigger validation and saving, and render the appropriate templates.

4.2 Relationships

- Appointment has a required foreign key to Service, and an optional foreign key to Barber.
- AppointmentForm creates and saves Appointment objects; ContactForm creates and saves ContactMessage objects.
- appointment_list depends on Appointment to retrieve every stored booking for the schedule view.